

Sécurité — Reminder Famille

Application des recommandations OWASP Top 10 sur les endpoints du back. Chaque section : risque expliqué, mesure prise, extrait de code défensif et fichier:ligne pour référence.

1. Injection SQL (OWASP A03:2021)

Risque : si une entrée utilisateur (ex: email, ID) est concaténée dans une requête SQL, un attaquant peut injecter du SQL malveillant.

Exemple d'injection (à NE PAS faire) :

```
// VULNÉRABLE
const sql = `SELECT * FROM users WHERE email = '${req.body.email}'`;
// Si email = "x' OR '1'='1" → toute la table est retournée
```

Mesure prise : 100% des requêtes SQL utilisent des **placeholders** ? (prepared statements). Le driver `mysql2` échappe correctement chaque valeur avant exécution.

Exemple de code défensif :

```
// back/src/models/user.model.js
async findByEmail(email) {
  const [rows] = await pool.query(
    'SELECT * FROM users WHERE email = ?',
    [email]
  );
  return rows[0] || null;
}
```

Vérification : `grep -r "pool.query\|pool.execute" back/src/` → 57 requêtes, **aucune** concaténation d'entrée utilisateur trouvée.

2. Faille XSS — Cross-Site Scripting (OWASP A03:2021)

Risque : un attaquant injecte du JavaScript dans un champ (titre de tâche, commentaire) qui s'exécute chez les autres utilisateurs.

Mesure prise : React **échappe automatiquement** tout texte rendu via les accolades JSX `{...}`. Pour qu'une chaîne soit interprétée comme du HTML, il faudrait utiliser explicitement `dangerouslySetInnerHTML` — **jamais utilisé** dans ce projet.

Exemple de code défensif :

```
// front/src/components/tasks/TaskCard.jsx
<h3>{task.title}</h3>      {/* échappé automatiquement */}
<p>{task.description}</p>  {/* échappé automatiquement */}
```

Même un titre `"<script>alert('XSS')</script>"` apparaîtra littéralement comme du texte, sans exécution.

Renforcement supplémentaire : `Content-Type: application/json` strict sur l'API → un navigateur ne traitera jamais une réponse API comme du HTML.

3. CSRF — Cross-Site Request Forgery (OWASP A01:2021)

Risque : un site malveillant force le navigateur de la victime à envoyer une requête à notre API en exploitant les cookies de session de la victime.

Mesure prise : l'authentification utilise un **token JWT dans le header `Authorization: Bearer`**, pas un cookie. Les requêtes cross-origin n'envoient pas automatiquement ce header → CSRF impossible.

Renforcement :

```
// back/src/app.js - CORS limité au front configuré
app.use(cors({
  origin: env.frontOrigin,      // ex: http://localhost:5173
  credentials: false,          // pas de cookie
}));
```

4. Hashage des mots de passe (OWASP A02:2021)

Risque : si la base est volée, les mots de passe doivent être inexploitable.

Mesure prise : **bcrypt avec coût 10** (~100ms par hash sur CPU moderne). Algorithme conçu pour rester lent même en cas d'attaque GPU/ASIC.

Différence hash vs chiffrement (point souvent demandé par jury) :

	Hash	Chiffrement
Réversibilité	❌ Non	✅ Oui (avec clé)
Usage	Vérifier un mdp	Stocker un secret à relire
Pour les mdp	✅ bcrypt, scrypt, argon2	❌ Jamais

Code défensif :

```
// back/src/services/password.service.js
export async function hashPassword(plain) {
  return bcrypt.hash(plain, 10);
}
export async function comparePassword(plain, hash) {
  return bcrypt.compare(plain, hash);
}
```

Le `password_hash` n'est **jamais retourné par l'API** : à chaque envoi au front on destructure :

```
const { password_hash, ...safe } = user;
res.json({ token, user: safe });
```

5. Contrôle des accès (OWASP A01:2021)

Risque : un enfant pourrait modifier les tâches d'autres familles, un non-membre pourrait voir les données d'une famille à laquelle il n'appartient pas.

Mesure prise : **middlewares composables en chaîne** sur chaque route :

```
authRequired
  → requireFamilyMember
  → requireAdmin (si action admin)
```

Chaque middleware fait **une seule chose** et `throw` une `HttpError` spécifique si la condition n'est pas remplie.

Exemple :

```
// back/src/routes/families.routes.js
router.patch('/:familyId/members/:userId',
  asyncHandler(requireFamilyMember), // ↳ 403 si pas membre
  requireAdmin,                      // ↳ 403 si pas admin
  asyncHandler(familiesController.updateMember));
```

Last-admin protection : impossible de retirer le dernier admin.

```
if (target.is_admin) {
  const admins = await familyMemberModel.countAdmins(familyId);
  if (admins <= 1) throw badRequest('Impossible de retirer le dernier
  administrateur');
}
```

Permissions sur tâches familiales :

- Création/modif/suppr : **parent uniquement**
- Complétion : **parent (toutes) OU enfant si `assigned_to === user.id`**
- Validation `pending_review` → `completed` : **parent uniquement**

6. Brute-force / Énumération (OWASP A07:2021)

Risque :

- Tester des millions de mots de passe sur `/auth/login`
- Distinguer « email inconnu » vs « mauvais mot de passe » pour énumérer les comptes existants

Mesures prises :

1. **Rate limiting** avec `express-rate-limit` :

```
// back/src/middleware/rateLimit.js (production)
export const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,    // 15 min
  max: 5,                      // 5 tentatives
  message: 'Trop de tentatives – réessayez dans 15 minutes',
});
```

2. Message d'erreur générique sur login :

```
// back/src/controllers/auth.controller.js
const user = await userModel.findByEmail(email);
if (!user) throw unauthorized('Identifiants incorrects'); // ← même msg
const ok = await comparePassword(password, user.password_hash);
if (!ok) throw unauthorized('Identifiants incorrects');    // ← même msg
```

→ Impossible de savoir si c'est l'email ou le mot de passe qui est faux.

7. Gestion des erreurs

Risque : retourner une stack trace en production révèle des chemins de fichiers, versions de bibliothèques, structure interne.

Mesure prise : `errorHandler` middleware central qui formate toutes les erreurs.

```
// back/src/middleware/errorHandler.js
export function errorHandler(err, req, res, next) {
  const status = err.status || 500;
  const body = { error: err.message || 'Erreur serveur' };
  if (err.details) body.details = err.details;
  if (status === 500) console.error('[500]', err); // log côté serveur uniquement
  res.status(status).json(body);
}
```

→ Le client ne voit **jamais** la stack trace. Les erreurs 500 sont loggées côté serveur pour debug.

8. Validation côté front ET back

Risque : la validation front est juste UX — un attaquant peut envoyer n'importe quel payload via curl ou Postman.

Mesure : double validation systématique.

Couche	Outil	Rôle
Front	<input required>, regex JS	UX immédiate (rouge sous le champ)
Back	validators/*.validator.js	Sécurité réelle : rejette HTTP 400

Exemple validator back :

```
// back/src/validators/auth.validator.js
export function validateRegister(body) {
  const errors = {};
  const { email, password, first_name, last_name } = body || {};
  if (!email || !EMAIL_RE.test(email) || email.length > 255)
    errors.email = 'Email invalide';
  if (!password || password.length < 8)
    errors.password = 'Mot de passe d\'au moins 8 caractères';
  // ...
  if (Object.keys(errors).length) throw badRequest('Champs invalides', errors);
  return { email, password, first_name, last_name };
}
```

→ Le back **ne fait jamais confiance** aux données client.

9. Secrets et configuration

Secret	Stockage	Jamais commit
DB_PASSWORD	back/.env	✓ .gitignore
JWT_SECRET	back/.env	✓ .gitignore
bcrypt hashes	base de données	jamais loggés

Vérification : `git log --all --source -- back/.env` → fichier jamais commit. Un `.env.example` est versionné avec des valeurs factices pour documenter les variables attendues.

10. Tokens d'API (calendar_token)

Risque : le token URL `/api/calendar/<token>.ics` est accessible sans JWT. S'il fuit (logs proxy, historique navigateur), un attaquant peut lire les tâches.

Mesures :

- Token = 48 caractères hex (24 octets aléatoires) — entropie 192 bits, brute-force inenvisageable
- Accès **read-only** : pas de mutation possible avec le token
- L'utilisateur peut régénérer le token via la page Profil (futur — actuellement régénération manuelle en base)

✓ Checklist OWASP — récapitulatif

Risque OWASP 2021	Mesure	Statut
A01 — Broken Access Control	Middlewares composables, last-admin protection	✓
A02 — Cryptographic Failures	bcrypt cost 10, JWT signé HS256	✓
A03 — Injection	100% requêtes préparées	✓

A04 — Insecure Design	Architecture en couches, séparation des responsabilités	✓
A05 — Security Misconfig	CORS limité, NODE_ENV strict	✓
A06 — Vulnerable Components	npm audit régulier, deps à jour	⚠ à automatiser
A07 — Auth Failures	Rate limit, message générique, mdp 8+ chars	✓
A08 — Software & Data Integrity	Pas de eval/exec, source contrôlée	✓
A09 — Logging	Erreurs serveur loggées, infos sensibles jamais loggées	✓
A10 — SSRF	Aucun appel HTTP server-side vers URL utilisateur	N/A

🎯 À retenir pour le jury

1. **Aucune injection SQL possible** — démontrable par grep
2. **React = anti-XSS gratuit** tant qu'on évite `dangerouslySetInnerHTML`
3. **JWT en header = pas de CSRF** (contrairement aux sessions cookie)
4. **bcrypt ≠ chiffrement** : irréversible, lent par design
5. **Double validation** : front pour UX, back pour sécurité réelle
6. **.env jamais commité**, vérifiable dans `git log`